

Homework 5

Week 9

1、给定平面上 n 个点 (x_i, y_i) , $i = 1, 2, 3, \dots, n$, 极大点定义为: 若 (x_i, y_i) 满足 $\forall j \neq i, (x_j < x_i) \vee (y_j < y_i)$, 则 (x_i, y_i) 为极大点
设计算法求给定点集的极大点, 并给出时间复杂度

解法一:

按照 x 坐标从大到小排序 (x 坐标相同的点, 按照 y 坐标从大到小排序)
之后扫描线从右向左扫描, 过程中维护 y 坐标的最大值 (初值尽可能小), 若 y 坐标最大值有更新, 则将该点加入解集中

时间复杂度取决于排序算法, 最快为 $O(n \log n)$

解法二:

求出凸包, 之后找到凸包上 x 坐标最大的点和 y 坐标最大的点 (如有多个, 则取另一个坐标尽可能大者), 从 x 坐标最大的点出发, 逆时针前进到 y 坐标最大的点, 路径上的所有点即为极大点

时间复杂度取决于求凸包算法, 最快为 $O(n \log n)$

2、(1)求凸 $n + 2$ 多边形不同三角划分种类数 (写出递推式即可)
(2)设计算法, 求三角划分对角线长度和的最小值

解:

(1)选择 1 条边, 以及不与该边相关联的任意 1 个顶点, 可将多边形划分为 3 部分, 这 3 部分分别拥有原多边形的数量为 $i, n - 1 - i, 1$

设 C_n 表示将凸 $n + 2$ 多边形划分为 n 个三角形的种类数, 则有递推式

$$C_0 = 1$$
$$C_n = \sum_{i=1}^n C_{i-1} C_{n-i}$$

C_n 为卡特兰数, 推导过程详见 CS2304 计算机科学中的数学基础 第2章组合数学 第3节生成函数, 解析解为

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

(2)动态规划

- $dp[i, j]$ 为 v_i 到 v_j 的所有边加上边 (v_j, v_i) 组成的多边形的三角划分中最小对角线长度和 (i 到 j 的部分按照 $1, 2, \dots, n$ 的循环队列增序依次排列)
- 初值:

$$dp[i, j] = \begin{cases} 0, & i = j \vee (i + 1) \% n = j \\ |v_i, v_j|, & (i + 2) \% n = j \end{cases}$$

- 迭代:

$$dp[i, j] = \min_k \{ dp[i, k] + dp[k, j] + |v_i, v_k| + |v_k, v_j| \}$$

$$k \in \begin{cases} (i, j), & i < j \\ (i, n] \cup [1, j), & i > j \end{cases} \wedge k \in \mathbb{N}_+$$

- 问题解: 满足 $(j + 1) \% n = i$ 的 $dp[i, j]$ 均为问题解

3、给定平面内 n 条线段，设计算法判断其中有无线段间交点，要求时间复杂度 $O(n \log n)$

解:

- 算法与求 n 条线段所有交点的算法相同，只是遇到交点就结束
- 但由于遇到交点就结束，所以时间复杂度有所降低:
 - 事件点数 $\leq 2n = O(n)$
 - 单个事件状态数 $\leq \log 2n = O(\log n)$
 - 总时间复杂度 $O(n \log n)$

4、利用扫描线算法求解最近点对问题

解:

先将各点按 x 坐标从小到大 (x 坐标相同的点按 y 坐标从小到大排序)

维护扫描过程中的最近距离，初始化为 $|v_1, v_2|$

扫描每个点时的事件处理也为扫描线算法

- 从左向右扫描各个点，并维护最小距离
- 若扫描过程中扫描线与始点的 x 坐标差已大于当前最小距离，则提前结束事件点

5、给定 $A[1..n]$ ， $A[i]$ 表示 $x = i$ 处木板高度，任意 2 块木板与 x 轴可构成容器，设计算法求单个容器的最大容积

解:

初始化 $left = 1$, $right = n$, $res = \min\{A[1], A[n]\} \times (n - 1)$

之后对 $A[left]$ 或 $A[right]$ 作出处理，处理步骤为:

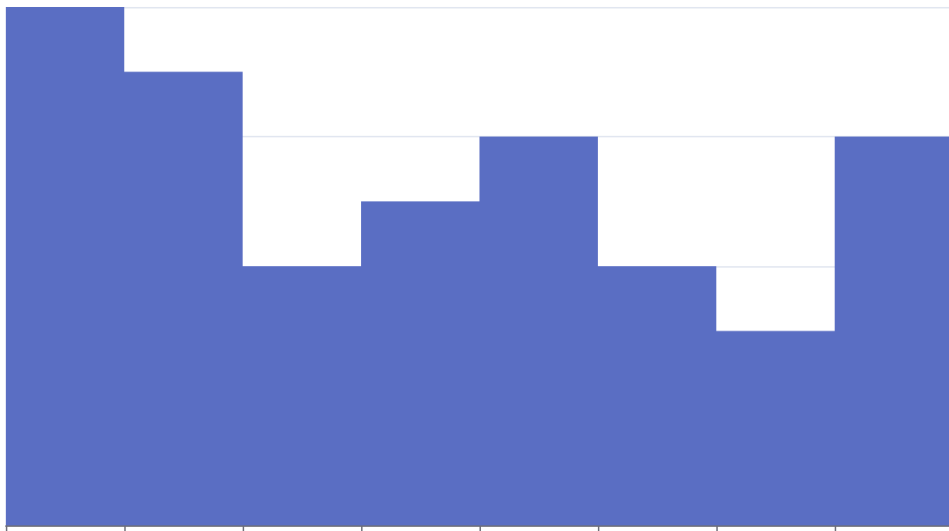
- 选择 $A[left]$ 或 $A[right]$ 中较小的一个 $A[move]$

- 暂存该侧木板高度的当前值 tmp ，每次向中间 ($left++$ 或 $right--$) 移动 1，并比较移动后的 $A[move]$ 与 tmp
 - 若 $A[move] \leq tmp$ ，则继续移动
 - 否则重新计算 $\min\{A[left], A[right]\} \times (n-1)$ 并更新 res
- 直到 $left = right$
- 时间复杂度：
 - 移动次数： $n = O(n)$
 - 事件点处理： $O(1)$
 - 总时间复杂度 $O(n)$

Week 10

1、海报墙由高为 $h[1..n]$ 的 n 块木板组成，木板的宽度均为 1

求海报墙上能粘贴矩形海报的最大面积，要求海报的边必须与木板侧边平行或垂直，给出算法及其时间复杂度



解：

依次求 $left[1..n]$, $right[1..n]$ ，其含义为：从木板 i 开始，若左侧或右侧木板不低于木板 i ，则向左或向右移动，直到遇到低于木板 i 的木板，将其下标对应保存到 $left[i]$, $right[i]$

维护方式：设立单调递增栈 s ，其元素存放木板下标

- 从左向右扫描木板
 - 若 $h[i] > h[s.top]$ ，则 $left[i]$ 为 $s.top + 1$ ，并将 i 入栈
 - 若 $h[i] \leq h[s.top]$ ，则 $s.top$ 出栈，并再次比较 $h[i]$ 与 $h[s.top]$

- 清空上述过程的栈
- 从右向左扫描木板
 - 若 $h[i] > h[s.top]$, 则 $right[i]$ 为 $s.top - 1$, 并将 i 入栈
 - 若 $h[i] \leq h[s.top]$, 则 $s.top$ 出栈, 并再次比较 $h[i]$ 与 $h[s.top]$
- 为便于实现上述维护, 可设 $h[0] = h[n+1] = 0$

最后, 遍历数组并求出 $\max_{1 \leq i \leq n} (right[i] - left[i] + 1) \times h[i]$ 即为问题的解

由于每个元素入栈、出栈最多 1 次, 所以维护 $left[1..n]$, $right[1..n]$ 的时间复杂度均为 $O(n)$, 总时间复杂度 $O(n)$

2、 给定数组 $Enve[1..n]$, $Enve[i] = (w_i, h_i)$ 表示第 i 个信封的宽度和高度; 当信封 a 的宽度和高度都比信封 b 大时, 可将信封 b 装入信封 a , 此时还允许信封 a 内含其他信封、信封 b 装入其他信封 (即“套娃”)

设计算法求最多能有多少个信封嵌套在一起

解:

将 $Enve[1..n]$ 按照 w_i 升序 (w_i 相同的按 h_i 降序) 排序, 之后求 $h[1..n]$ 序列的最长递增子序列即可

3、 求 $n!$ 包含质因子 p 的次数, 设计算法并给出时间复杂度

解:

```

init res = 0, k = 0
while (n ≥ pk) res = res + ⌊ $\frac{n}{p^k}$ ⌋, k = k + 1
return res

```

时间复杂度 $O(\log_p n) = O(\log n)$, 由于输入规模 $O(\log n)$, 所以该算法为线性时间

4、 证明: $\forall n \in \mathbb{N} \wedge n \geq 2$, n 可表示为至多 $\log_2 n$ 个 *Fibonacci* 数之和, 并设计算法求上述表示方式

解:

易知 $Fib(n) > Fib(n-1) > Fib(n-2)$

$\therefore Fib(n) = Fib(n-1) + Fib(n-2)$

$\therefore Fib(n) < Fib(n-1)$

$\therefore$ 小于 n 的最大 *Fibonacci* 数 $Fib(k) > \frac{n}{2}$

设 $< \frac{n}{2}$ 的整数可用 $< \log_2 \frac{n}{2} = \log_2 n - 1$ 个 *Fibonacci* 数之和表示

则 $n - Fib(k) < \frac{n}{2}$ 个 *Fibonacci* 数之和表示

$\therefore n$ 可用上述 $\log_2 n - 1$ 个数加上 $Fib(k)$ 表示

由数学归纳法可知成立, 算法设计与证明过程类似

5、用线性时间算法求 *Fibonacci* 数列的第 n 项即 $Fib(n)$ ，假设不支持浮点数操作

解：

$$\begin{aligned}\begin{pmatrix} Fib(n) \\ Fib(n-1) \end{pmatrix} &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} Fib(n-1) \\ Fib(n-2) \end{pmatrix} \\ &= \dots \\ &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-2} \begin{pmatrix} Fib(2) \\ Fib(1) \end{pmatrix}\end{aligned}$$

关键在于求 $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-2}$ ，可用分治法计算

时间复杂度 $O(\log n)$ ，与输入规模同阶，为线性时间